

What we're building, and why it's shaped this way

A walk through the whole system — the programme it runs, the data behind it, the rules the database enforces, and how we'll build it. **No prior knowledge assumed.**

28

database tables

23

screens designed

10

user flows mapped

3

types of user

Navigate with   or the buttons below.

We run a free training programme that takes students from college into jobs.

Colleges send us candidates. We interview and pick the best. We train them for about a year. If they attend enough, they get certified. Then we help them get hired. **Everything in the system exists to run or record part of that.**

WHO SENDS STUDENTS

Colleges

Partner institutions. They host the classes on their own campus and give us candidates.

WHO WE SERVE

Students

Candidates from low-income families — under ₹4 lakh a year. They pay a refundable deposit, nothing more.

WHO TEACHES

Trainers

Paid per hour taught. JavaScript, HTML, CSS, Zoho Creator and more.

WHO SELECTS

Panel members

External interviewers who score candidates at entrance. Also paid per hour.

WHO HIRES

Companies

Employers who interview our graduates and make offers.

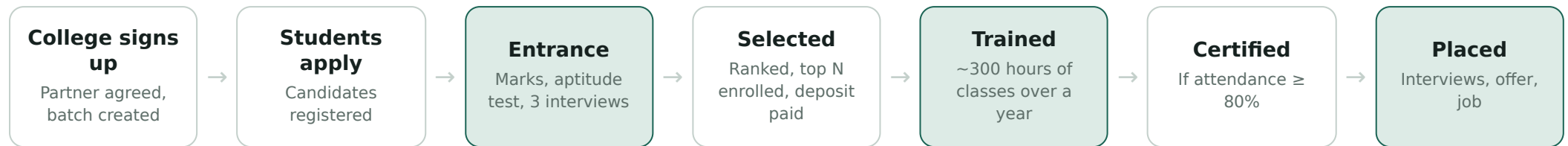
AND US

The coordinators

Staff who run all of it — and the main users of the app we're building.

One student's path through the system

Every screen we build sits somewhere on this line. When you're lost, come back to it.



Running alongside the whole time: trainers and panel members record their hours, and get invoiced and paid monthly. That's a second track through the same system.

Three kinds of user — and three very different screens

Students, colleges and panel members have **no logins**. This is an internal tool for staff. That single decision shapes half the design.

DESKTOP

Admin / Coordinator

Runs everything. 14 of our 23 screens.

- Colleges, batches, candidates
- Entrance scoring and selection
- Curriculum and timetable
- Certificates and billing

PHONE FIRST

Trainer

Uses it standing in a classroom. 5 screens, big buttons.

- My schedule today
- Start/end a class
- Take attendance
- Enter test scores
- See my invoices

DESKTOP

Placement officer

Only the job-hunting part. 3 screens.

- Companies and open roles
- Interview rounds
- Offers and salaries

Why does this matter so early? Because **"who can see this?" changes every query we write**. A trainer must only ever load their own sessions and their own invoices.

Every important noun becomes a table

That's genuinely the whole trick. Read the story back, underline the nouns, and each one gets a table with columns for the facts you need about it.

"St. Mary's College"

becomes a row in

colleges

"The 2026-27 group"

becomes a row in

batches

"Ananya, a candidate"

becomes a row in

students

"Monday 9am JavaScript class"

becomes a row in

class_session

"Ananya was present"

becomes a row in

session_attendance

"June invoice for Priya"

becomes a row in

billing

Notice the last two. **Events and records are nouns too** — "an attendance mark", "an invoice". Beginners usually spot people and places, then miss the events. Most of our 28 tables are events.

You already know most of this — different words

The concepts carry over almost one to one. What changes is that the database itself will enforce far more of the rules, instead of your scripts doing it.

CREATOR

Form

Where you define fields and collect records.

DATABASE

Table

Same idea. `colleges` is a form; each college is a record/row.

CREATOR

Field

Single line, number, date, decision box...

DATABASE

Column

`Text`, `Integer`, `Date`, `Boolean`. Same types, plainer names.

CREATOR

Lookup field

Points a record at another form's record.

DATABASE

Foreign key

Exactly this. `batches.college_id` points at a row in `colleges`.

CREATOR

Deluge validation

Your script checks the rule when the form is submitted.

DATABASE

Constraint

The rule lives in the table itself. **It holds even if someone bypasses the app.** This is the big upgrade — next few slides.

One-to-many: the "many" side holds the link

One college runs many batches. Ask yourself: where do we store the connection?



The rule: the "many" side stores the ID of the "one" side. Each batch remembers its college. The college doesn't keep a list of batches — you'd never know how many slots to leave. To get a college's batches you ask: give me all batches where `college_id = this one`.

This same shape repeats everywhere: a batch has many students, a module has many topics, an invoice has many line items.

Many-to-many needs a table in the middle

Real example from our system: **one subject is taught by several trainers, and one trainer teaches several subjects.** Now neither side can hold the link.

DOESN'T WORK

Put `trainer_id` on the subject

JavaScript can then only have one trainer. But Priya and Arun both teach it. We had this bug in the first draft of the schema — it's why the column was removed.

WORKS

A third table: `subject_trainer_mapping`

Each row says one small fact: *"Priya teaches JavaScript in batch 2026-27."* Want two trainers? Two rows. Want three? Three rows. No limit, no empty columns.

subjects

JavaScript



subject_trainer_mapping

subject + trainer + batch, one row per fact



trainers

Priya, Arun

How to spot it: if you can say "many X have many Y", you need the middle table. Always.

Four columns appear on all 28 tables

Learn these once and 28 tables get much less intimidating.

IDENTITY

id

A unique code for the row. We use UUIDs — long random values, never reused, never guessable.

SOFT DELETE

active

true/false. Setting it false **hides** a row instead of destroying it.

AUDIT

created_at

When the row was made. Never changes.

AUDIT

updated_at

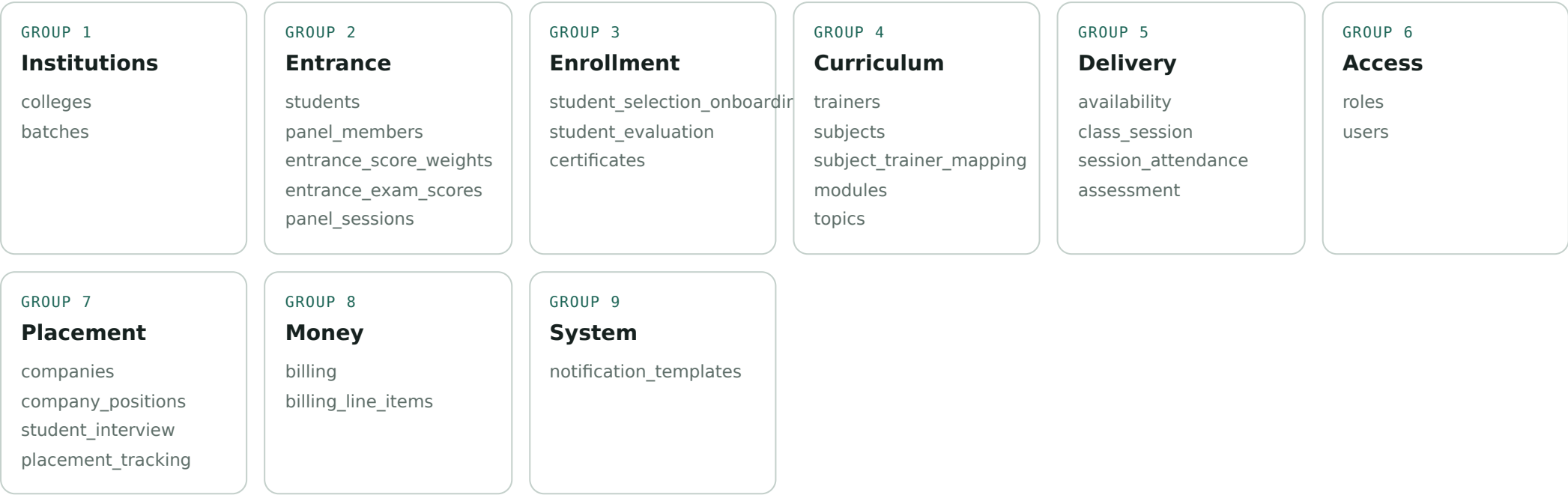
When it last changed. Answers "who changed this and when?"

WHY WE NEVER TRULY DELETE ANYTHING

Suppose a trainer leaves and you delete them. Every invoice you ever paid them now points at nothing. Last year's classes have no teacher. Your accounts break retroactively. **So we set `active = false` instead:** they vanish from dropdowns and lists, while history stays intact and honest.

28 tables, in nine groups

Nobody memorises 28 tables. People remember nine groups, then look up the detail.



Curriculum nests three deep: **Subject** → **Module** → **Topic**. "JavaScript" → "JS Syntax" (12 hrs) → "JS Statements" (1.5 hrs). A topic is what one class actually teaches.

Let the database say no.

You could check every rule in your app code. Most beginners do. Here's why we don't rely on that alone — and this is the single biggest idea in the briefing.

RULES ONLY IN APP CODE

Works — until someone imports a spreadsheet, runs a fix-it script, uses a second app, or you simply forget the check on one screen out of twenty. **Then bad data is already saved**, and you find out months later from an accountant.

RULES IN THE DATABASE

The rule sits on the table. Every route in — app, script, import, console — hits the same wall. **Invalid data cannot physically be stored.** You get an error immediately, not a mystery next quarter.

Both, ideally: check in the app for a friendly message, **and** in the database so it's actually guaranteed. The app's job is a nice error. The database's job is the truth.

What that looks like in practice

ENTRANCE SCORING

The weights must total exactly 1

Marks, attendance, aptitude, HOD, and 3 panelists are weighted. If they sum to 0.95, every candidate is quietly under-scored and rankings are wrong. The table refuses to save unless the seven add to 1.0000.

ATTENDANCE

One mark per student per class

Without this, a student can be marked present twice and absent once for the same session. Which is true? The database allows exactly one row per student per class.

BILLING

A class can be billed only once

Generate June's invoice twice and you'd pay the same class twice. Each session can appear on **one** invoice line, ever. Real money, protected by one line of schema.

CERTIFICATION

No certificate below 80% attendance

The rule is a promise to employers, so it can't depend on someone remembering to check. Attendance is computed from real marks; below the line, issuing is blocked.

Notice the pattern: each rule protects something expensive — fair rankings, honest records, real money, our reputation. That's how you decide what deserves a constraint.

Copy the number onto the invoice. Don't look it up later.

Priya teaches for us at ₹800/hour in June. In July, she gets a raise to ₹1,000/hour. What should June's invoice say?

IF THE INVOICE READS HER CURRENT RATE

June's invoice was 8 hrs × ₹800

₹6,400 → ₹8,000

The moment you save her raise, **every past invoice silently rewrites itself**. Paid, closed months change value. Your books stop matching your bank.

IF WE COPY THE RATE ONTO THE LINE

The line stores `rate_applied = 800`

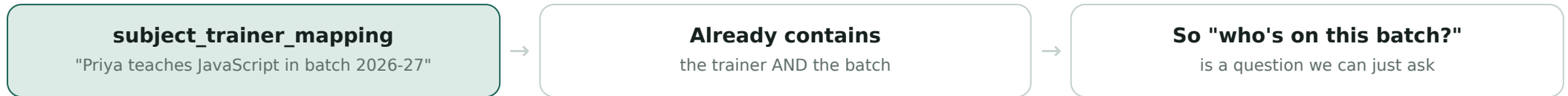
₹6,400 → ₹6,400

June stays ₹6,400 forever. July uses ₹1,000. **History is frozen; the future moves on.**

The general rule: when a number is part of a completed transaction — a price, a rate, a tax percentage — store a copy of it *on that transaction*. Look values up live only when you want them to change. This one habit prevents a whole category of accounting bugs.

If you can work it out, work it out

We nearly shipped a `trainer_batch` table listing which trainers work on which batch. We deleted it. Here's why.



The danger of keeping both: remove Priya from the batch list but leave her subject rows, and now the system holds two contradictory answers to one question. Whichever screen you happen to be on decides what you believe. Nothing errors — it just quietly disagrees with itself.

RULE OF THUMB

One fact, one place. If two tables can ever disagree, delete one of them and calculate it instead. **The exception is the previous slide** — a deliberate frozen copy for history. Know which one you're doing and why.

23 screens, already designed and annotated

Every wireframe lists the exact fields it reads and writes — so building a screen is mostly translation, not invention.

MOSTLY GENERATED

Simple list + form

Colleges, companies, positions, users, curriculum. A list, a search box, an add/edit form. Fast to build, low risk.

HAND-BUILT

The clever ones

Entrance scoring (locks until all 3 panelists are in), the timetable mapper, phone attendance, invoice generation. **This is where the real work is.**

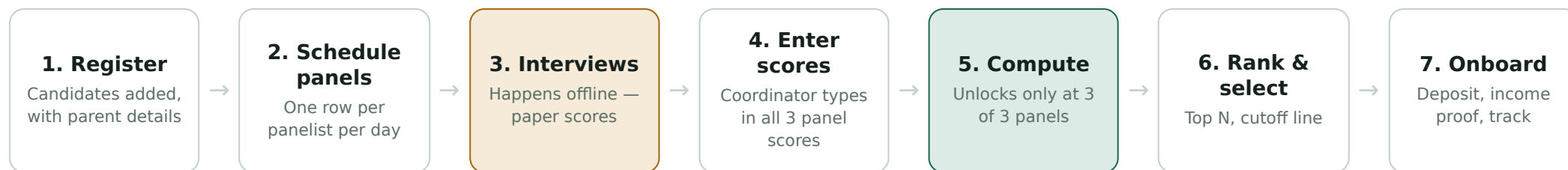
PHONE

Trainer screens

Used one-handed in a noisy classroom. Big tap targets, one decision per row, no typing where a tap will do.

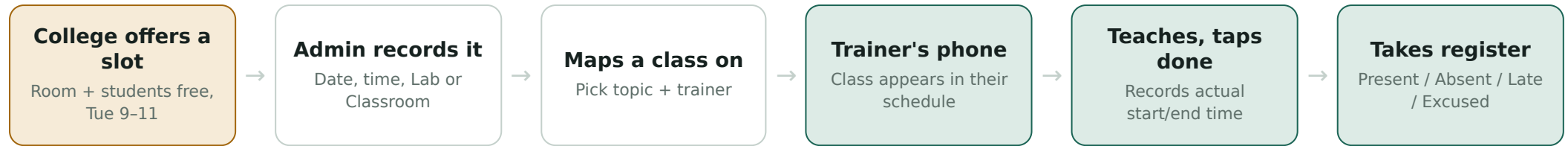
A design principle we used everywhere: if the database forbids something, the screen shouldn't offer it. The "Issue certificate" button is disabled below 80%. The timetable only lists trainers who actually teach that subject in that batch. **You can't make the mistake, so you never see the error.**

Entrance: from applicant to enrolled student



Amber = happens in the real world, not in the app. Worth marking, because it's where data gets delayed or lost. Panelists interview on different days, so scores arrive piecemeal — the app saves partial rows happily, but refuses to *compute a total* until all three are in. Half a panel must never produce a ranking.

Running a class, and marking who came



WHY "ACTUAL" TIMES, NOT PLANNED

The slot said 9:00–11:00. The class ran 9:05–11:00. **We pay for what happened, not what was scheduled.** So the trainer records real times, and billing reads those.

WHY THE REGISTER CAN'T BE HALF-DONE

Submit stays disabled until **every** student is marked. A partly-filled register is worse than none — it looks complete while quietly under-counting someone's attendance, which decides their certificate.

How a taught hour becomes a paid invoice



Every rupee traces back to a specific class on a specific day. If Priya questions her invoice, we show her the exact sessions on it. That's why line items exist at all — the original design just had a total, which is impossible to defend or debug.

TWO KINDS OF PAYEE, ONE SYSTEM

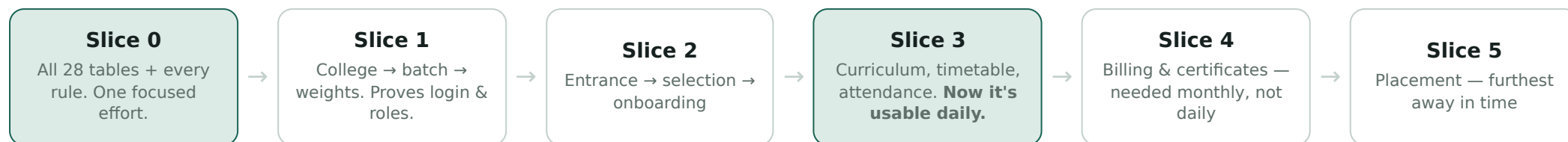
Trainers bill for classes; panel members bill for interview sittings. Same invoice tables. A rule guarantees each invoice has **exactly one** payee — never zero, never both.

ATTENDANCE NEVER AFFECTS PAY

A trainer is paid for teaching, whether 30 students came or 4. The attendance chain and the money chain are deliberately kept apart.

Thin slices, not layers

The instinct is to build all the tables, then all the screens, then all the logic. Don't. **Build one complete journey at a time** — each slice is demoable and actually usable.



Why this order: it follows the student's journey, so each slice is something real you can show a coordinator and get corrections on early. Billing can wait — the first invoice isn't due until a month after teaching starts. Placement can wait a year.

Three decisions we need before writing tables

Everything else is settled. These three change the schema, and schema changes are cheapest **before** there's data in it.

QUESTION 4

Does the Zoho vs non-Zoho track change what's taught? Right now nothing connects a student's track to a set of subjects. If the two tracks study different things, we need another link — and the timetable must filter by it.

QUESTION 7

Does the system generate certificate PDFs, or just record that one was issued? We store a template link but have nowhere to put the finished certificate. If we generate, we need a column for the output file.

QUESTION 10

What happens to the deposit if a student drops out? Refunds are defined for students who complete. Dropouts are undefined — and dropouts are exactly when someone will ask for their money back.

There are ten open questions in total; the other seven affect screens, not tables, so they can be answered while building.

Five things to remember

1 • Nouns become tables

Including events — "a class happened", "a student was present", "an invoice was raised".

2 • The many side holds the link

And when both sides are "many", you need a table in the middle.

3 • Let the database say no

Rules on the table hold no matter how the data arrives. App checks are for friendly messages.

4 • Freeze money, calculate the rest

Copy rates onto invoices. Everything else, work out from one source of truth.

5 • Never delete — deactivate

History has to stay standing, or your past stops making sense.

WHERE EVERYTHING LIVES

The repo

[APP-Data-Structure-Detailed.md](#) — all 28 tables, every field explained.

[APP-Wireframes-Storyboards.html](#) — 23 screens, 10 flows.

Questions now — and if something in the schema looks wrong to you, **say so**. Six real bugs were caught by exactly that kind of second look.